

C++ 期末复习知识点整理

基于《期末复习2-2 1.pdf》整理：通俗讲解 + 代码举例

说明：本资料按课件中出现的知识点逐项整理。每个大模块都包含“核心概念、通俗理解、易错点、代码例子”。代码以 C++ 为主，适合期末选择题、改错题、读程序写结果、填空题和程序设计题复习。

资料范围对应课件第 4 页：类和对象、静态成员与常量成员、运算符重载；继承、虚拟继承、多态、虚函数、纯虚函数与抽象基类；函数模板、类模板、链表、栈、队列、STL；输入输出流、格式控制、文本文件和二进制文件读写。

□

一、考试题型与复习范围

来源页码：第 2-4 页

1. 考试题型

- 单项选择：通常考概念、语法细节、访问权限、构造顺序、模板实例化、文件读写函数等。
- 程序改错：常见错误包括类定义后漏分号、成员函数类外定义漏“类名::”、构造函数初始化错误、文件未打开、深浅拷贝错误等。
- 读程序写结果：重点看构造/析构输出顺序、虚函数动态绑定、运算符重载调用、模板类型推导、文件读写结果。
- 程序填空：常填构造函数初始化列表、operator 函数头、模板声明、文件流打开方式、read/write 参数。
- 程序设计：常见为自定义类、运算符重载、继承多态、链表模板、文件输入输出综合题。

2. 总复习范围速览

模块	课件关键词	复习目标
类和对象	类定义、访问权限、构造/析构、拷贝构造、委托构造、静态成员、常量成员、友元	会写类、会判断成员能否访问、会分析对象初始化和构造/析构顺序
运算符重载	成员函数方式、友元方式、赋值运算符、输入输出运算符	会写 operator 函数，能分辨参数个数和顺序，理解深拷贝
继承与多态	派生类、继承方式、多继承、多级继承、虚继承、虚函数、纯虚函数、抽象基类	会判断继承后的访问权限，能用基类指针调用派生类虚函数
模板与数据结构	函数模板、类模板、链表、栈、队列、STL 容器	会定义/实例化模板，理解 vector/list/map/set 等容器的使用场景
输入输出流与文件	cin/cout/cerr/clog、格式控制、文本文件、二进制文件、随机访问	会用流读写数据、控制输出格式、读写文本/二进制文件

□

二、类和对象

来源页码：第 6-8 页

1. 类的定义

类可以理解为“自己定义的一种数据类型”。它把数据（成员变量）和处理这些数据的函数（成员函数）放在一起。

比如“学生”这个类，可以有姓名、成绩等成员变量，也可以有打印信息、计算等级等成员函数。

- 类名是标识符，命名规则和变量名类似。
- 类由成员变量和成员函数组成。
- 类定义结束后必须加分号“;”。
- 成员函数如果写在类外，函数名前必须加“类名::”。

易错提醒：很多改错题会故意漏掉类定义后的分号，或漏掉类外成员函数定义的“类名::”。

```
#include <iostream>
using namespace std;

class Student {           // 类定义
private:
    string name;
    int score;
public:
    void setInfo(string n, int s); // 类内声明
    void print();
}; // 注意：类定义后必须有分号

void Student::setInfo(string n, int s) { // 类外定义要写 Student::
    name = n;
    score = s;
}

void Student::print() {
    cout << name << " " << score << endl;
}

int main() {
    Student stu;
    stu.setInfo("Li", 90);
    stu.print();
    return 0;
}
```

2. 类成员访问权限：public、private、protected

访问权限决定“谁能直接使用这个成员”。可以把类想成一个房间：public 是公开区域，谁都能进；private 是个人抽屉，只有类自己能打开；protected 是家族成员区域，类自己和派生类可以访问。

- public：类外可以访问。
- private：默认权限，类外不能访问；非友元、非继承情况下也不能访问。
- protected：类外不能访问，但派生类成员函数可以访问基类 protected 成员。
- 类内不仅指类定义体，也包括类的成员函数体。成员函数可以访问同类对象的 private 成员。

易错提醒：class 默认 private；struct 默认 public。考试中常问“类外能否访问私有成员”。答案通常是不能，除非通过公有成员函数或友元。

```
#include <iostream>
using namespace std;

class A {
private:
    int x = 1;
protected:
    int y = 2;
public:
    int z = 3;
    void show() {
        cout << x << " " << y << " " << z << endl; // 类内都能访问
    }
};

int main() {
    A a;
    // cout << a.x; // 错：private 类外不能访问
}
```

```
// cout << a.y; // 错: protected 类外不能访问
cout << a.z << endl; // 对: public 类外能访问
a.show();
return 0;
}
```

3. 构造函数与析构函数

构造函数用于“对象出生时初始化”，析构函数用于“对象死亡时清理资源”。构造函数名与类名相同，没有返回值；析构函数名前加~，也没有返回值，且一个类只能有一个析构函数。

- 对象创建时自动调用构造函数。
- 对象生命周期结束时自动调用析构函数。
- 构造顺序：成员对象和基类先构造，派生类后构造；析构顺序相反。
- 如果类中有 const 成员或引用成员，通常必须用初始化列表初始化。

```
#include <iostream>
using namespace std;

class Point {
private:
    int x, y;
public:
    Point(): x(0), y(0) { // 无参构造
        cout << "无参构造" << endl;
    }
    Point(int a, int b): x(a), y(b) { // 有参构造, 初始化列表
        cout << "有参构造" << endl;
    }
    ~Point() {
        cout << "析构" << endl;
    }
    void show() { cout << x << ", " << y << endl; }
};

int main() {
    Point p1; // 调用无参构造
    Point p2(3, 4); // 调用有参构造
    p2.show();
    return 0;
}
```

4. 初始化列表、拷贝构造函数、委托构造函数

初始化列表写在构造函数参数表后面，用冒号开头。它不是“赋值”，而是对象刚创建时直接初始化，因此更高效，也能初始化 const 成员、引用成员和对象成员。

- 拷贝构造函数通常形式：ClassName(const ClassName& obj)。当用已有对象初始化新对象、函数按值传参、函数按值返回对象时可能被调用。
- 委托构造函数是一个构造函数调用同类另一个构造函数，减少重复代码。

```
#include <iostream>
using namespace std;

class Box {
private:
    const int id; // const 成员必须初始化列表初始化
    int value;
public:
    Box(): Box(0, 0) {} // 委托构造: 调用 Box(int,int)
    Box(int i, int v): id(i), value(v) {}
    Box(const Box& b): id(b.id), value(b.value) {} // 拷贝构造
        cout << "copy constructor" << endl;
    }
    void show() const { cout << id << ": " << value << endl; }
};

int main() {
    Box b1(1, 100);
    Box b2 = b1; // 用已有对象初始化新对象, 调用拷贝构造
    b2.show();
}
```

```
return 0;
}
```

易错提醒：“对象初始化”和“对象赋值”不是一回事：Box b2 = b1 是初始化，通常调用拷贝构造；b2 = b1 是赋值，调用赋值运算符。

5. 静态成员

静态成员属于“类本身”，不专属于某一个对象。可以把 static 成员理解成全班共享的一块黑板，而普通成员是每个学生自己的笔记本。

- 静态数据成员通常要在类外定义一次。
- 静态成员函数没有 this 指针，只能直接访问静态成员，不能直接访问普通成员。
- 可以通过 类名::成员 访问，也可以通过对象访问。推荐用 类名::成员。

```
#include <iostream>
using namespace std;

class Student {
private:
    string name;
public:
    static int count; // 声明静态数据成员
    Student(string n) : name(n) { count++; }
    static void showCount() { // 静态成员函数
        cout << "学生人数: " << count << endl;
        // cout << name; // 错：静态函数不能直接访问普通成员
    }
};

int Student::count = 0; // 类外定义并初始化

int main() {
    Student a("A"), b("B");
    Student::showCount();
    return 0;
}
```

6. 常量成员：const 数据成员与 const 成员函数

常量成员在课件范围中被列为复习内容。const 数据成员一旦初始化后不能改；const 成员函数承诺“不修改对象的普通数据成员”。

- const 数据成员必须通过初始化列表初始化。
- const 成员函数写法：返回值 函数名(参数) const。
- const 对象只能调用 const 成员函数。

```
#include <iostream>
using namespace std;

class Circle {
private:
    const double pi; // const 数据成员
    double r;
public:
    Circle(double radius) : pi(3.14159), r(radius) {}
    double area() const { // const 成员函数，不修改对象
        return pi * r * r;
    }
};

int main() {
    const Circle c(2.0);
    cout << c.area() << endl; // const 对象可以调用 const 成员函数
    return 0;
}
```

7. 友元函数

友元不是类的成员，但被类“授权”，可以访问类的 private 和 protected 成员。它常用于运算符重载，尤其是需要让左操作数不是当前类对象的情况。

- 在类中用 friend 声明。
- 友元函数通常把类对象作为参数，通过对象访问私有成员。
- 友元破坏封装性，考试中要会写，但实际工程中应谨慎使用。

```
#include <iostream>
using namespace std;

class Account {
private:
    double balance;
public:
    Account(double b) : balance(b) {}
    friend void showBalance(Account a); // 声明友元
};

void showBalance(Account a) {
    cout << "余额: " << a.balance << endl; // 友元可访问 private
}

int main() {
    Account acc(1000);
    showBalance(acc);
    return 0;
}
□
```

✓ 友元写后在外部访问 private and protected

三、运算符重载

来源页码：第 9-11 页，第 32 页，第 44 页例题

1. 运算符重载的基本概念

运算符重载就是让已有运算符（如 +、-、>、=、<<、>>）能用于自定义类对象。比如两个 Time 对象相减，编译器本来不知道该怎么减；写了 operator- 后，就能按我们定义的规则相减。

- 成员函数方式：a + b 等价于 a.operator+(b)。调用对象 a 是第一个运算分量。
- 友元/普通函数方式：a + b 等价于 operator+(a, b)。所有运算分量都作为参数传入。
- 函数名写成 operator 运算符，如 operator+、operator>、operator=。
- 友元方式通常比成员函数方式多一个参数，因为成员函数隐含了 this。

易错提醒：表达式中运算分量的顺序必须和函数参数顺序一致。尤其是减法、比较运算符等非交换运算，顺序错了结果就错。

2. 成员函数方式重载运算符

当左操作数就是当前类对象时，可以用成员函数方式。例如 time2 - time1 中，time2 调用 operator-，time1 作为参数传入。

```
#include <iostream>
using namespace std;

class Time {
private:
    int h, m, s;
public:
    Time(int hh=0, int mm=0, int ss=0) : h(hh), m(mm), s(ss) {}

    // 成员函数方式：this 是左操作数，t 是右操作数
    Time operator-(const Time& t) const {
        int a = h * 3600 + m * 60 + s;
        int b = t.h * 3600 + t.m * 60 + t.s;
        int d = a - b;
        if (d < 0) d = -d; // 简化处理：取绝对差
        return Time(d / 3600, d % 3600 / 60, d % 60);
    }

    void show() const { cout << h << ":" << m << ":" << s << endl; }
};
```

```
int main() {
    Time t1(12,10,20), t2(13,15,25);
    Time d = t2 - t1; // 等价于 t2.operator-(t1)
    d.show();        // 1:5:5
    return 0;
}
```

3. 友元方式重载运算符

如果需要直接访问私有成员，又不希望把运算符写成成员函数，可以用友元方式。课件例题要求以友元方式重载“>”比较 Time 对象。

```
#include <iostream>
using namespace std;

class Time {
private:
    int h, m, s;
public:
    Time(int hh=0, int mm=0, int ss=0) : h(hh), m(mm), s(ss) {}

    friend bool operator>(const Time& a, const Time& b); // 友元声明
};
```

```
bool operator>(const Time& a, const Time& b) {
    int ta = a.h * 3600 + a.m * 60 + a.s;
    int tb = b.h * 3600 + b.m * 60 + b.s;
    return ta > tb;
}
```

```
int main() {
    Time t1(12,10,20), t2(13,15,25);
    cout << (t2 > t1) << endl; // 1, 表示 true
    return 0;
}
```

4. 赋值运算符重载与深拷贝

默认赋值通常是“浅拷贝”：把成员变量逐个复制。如果成员里有指针，浅拷贝只复制地址，两个对象会指向同一块内存，析构时可能重复释放，导致错误。因此有指针成员时通常要写析构函数、拷贝构造函数、赋值运算符，即“三法则”。

```
#include <iostream>
#include <cstring>
using namespace std;

class StringBox {
private:
    char* p;
public:
    StringBox(const char* s="") {
        p = new char[strlen(s) + 1];
        strcpy(p, s);
    }
    ~StringBox() { delete[] p; }

    // 深拷贝赋值运算符
    StringBox& operator=(const StringBox& other) {
        if (this == &other) return *this; // 防止自己给自己赋值
        delete[] p;
        p = new char[strlen(other.p) + 1];
        strcpy(p, other.p);
        return *this;
    }

    void show() const { cout << p << endl; }
};

int main() {
    StringBox a("hello"), b("world");
    b = a;
    b.show();
    return 0;
}
```

5. 输入/输出运算符重载

课件强调 `istream& operator>>` 和 `ostream& operator<<` 的返回值和参数类型。返回流引用是为了支持连续输入输出，如 `cout << a << b`。

```
#include <iostream>
using namespace std;

class Complex {
private:
    double real, imag;
public:
    Complex(double r=0, double i=0) : real(r), imag(i) {}
    friend istream& operator>>(istream& in, Complex& c);
    friend ostream& operator<<(ostream& out, const Complex& c);
};

istream& operator>>(istream& in, Complex& c) {
    in >> c.real >> c.imag;
    return in;
}

ostream& operator<<(ostream& out, const Complex& c) {
    out << c.real << "+" << c.imag << "i";
    return out;
}

int main() {
    Complex c;
    cin >> c;    // 输入：3 4
    cout << c << endl; // 输出：3+4i
    return 0;
}
```

四、继承与多态

来源页码：第 12-15 页，第 42 页例题

1. 派生类与继承方式

继承就是“在已有类基础上扩展新类”。已有类叫基类，新类叫派生类。例如 `Person` 是基类，`Student` 是派生类，`Student` 继承 `Person` 的姓名、年龄，并新增学号、成绩。

- 派生类声明格式：`class 派生类名 : 继承方式 基类名 { ... };`
- 单继承：一个派生类只有一个直接基类。
- 多重继承：一个派生类有多个直接基类。
- 多级继承：A 派生 B，B 再派生 C。
- 虚拟继承用于解决多重继承中的“菱形继承”重复基类问题。

2. 继承方式与访问权限

继承方式会改变基类 `public/protected` 成员在派生类中的身份。基类 `private` 成员无论哪种继承方式，派生类成员函数都不能直接访问。

基类成员 `public` 继承后 `protected` 继承后 `private` 继承后

<code>public</code>	<code>public</code>	<code>protected</code>	<code>private</code>
<code>protected</code>	<code>protected</code>	<code>protected</code>	<code>private</code>
<code>private</code>	不可直接访问	不可直接访问	不可直接访问

易错提醒：选择题中“私有继承是将基类所有成员都继承为派生类私有成员”这种说法不严谨，因为基类 `private` 成员虽然存在于派生类对象中，但派生类不能直接访问。

```
#include <iostream>
using namespace std;

class Base {
```

```
private:
    int a = 1;
protected:
    int b = 2;
public:
    int c = 3;
};

class Derived : public Base {
public:
    void show() {
        // cout << a; // 错: 基类 private 不能直接访问
        cout << b << " " << c << endl; // 对: protected 和 public 可访问
    }
};

int main() {
    Derived d;
    // cout << d.b; // 错: protected 类外不能访问
    cout << d.c << endl; // public 继承后仍为 public
    d.show();
    return 0;
}
```

3. 派生类构造函数与析构函数顺序

创建派生类对象时，必须先把“父类部分”构造好，再构造派生类自己的成员。析构时反过来，先拆派生类，再拆基类。

- 构造顺序：基类成员/基类对象 → 对象成员 → 派生类成员。
- 析构顺序：与构造顺序相反。
- 派生类构造函数要在初始化列表中调用基类构造函数，并初始化对象成员。

```
#include <iostream>
using namespace std;

class Base {
public:
    Base() { cout << "Base 构造" << endl; }
    ~Base() { cout << "Base 析构" << endl; }
};

class Member {
public:
    Member() { cout << "Member 构造" << endl; }
    ~Member() { cout << "Member 析构" << endl; }
};

class Derived : public Base {
private:
    Member m;
public:
    Derived() { cout << "Derived 构造" << endl; }
    ~Derived() { cout << "Derived 析构" << endl; }
};

int main() {
    Derived d;
    return 0;
}
// 输出顺序: Base构造 -> Member构造 -> Derived构造 -> Derived析构 -> Member析构 -> Base析构
```

4. 基类指针、虚函数与多态

多态的核心是：同一个调用语句，根据对象真实类型执行不同函数。实现条件通常是“基类指针/引用 + 虚函数 + 派生类重写”。

- 可以把派生类对象地址赋给基类指针。
- 只有基类中声明为 virtual 的成员函数，才会通过基类指针动态绑定到派生类版本。
- 若基类析构函数可能通过基类指针删除派生类对象，应把基类析构函数设为 virtual。

```
#include <iostream>
using namespace std;

class Animal {
public:
    virtual void speak() { cout << "Animal" << endl; }
    virtual ~Animal() {}
};

class Dog : public Animal {
public:
    void speak() override { cout << "Dog" << endl; }
};

int main() {
    Dog d;
    Animal* p = &d; // 基类指针指向派生类对象
    p->speak(); // 调用 Dog::speak, 实现多态
    return 0;
}
```

5. 纯虚函数与抽象基类

纯虚函数相当于“基类只规定接口，不给具体做法”。包含纯虚函数的类是抽象基类，不能创建对象，只能让派生类继承并实现。

- 纯虚函数写法：virtual 返回值 函数名(参数表) = 0;
- 抽象基类不能定义对象或对象数组。
- 派生类必须实现所有纯虚函数，否则派生类仍然是抽象类。

```
#include <iostream>
using namespace std;

class Shape {
public:
    virtual double area() const = 0; // 纯虚函数
    virtual ~Shape() {}
};

class Rectangle : public Shape {
private:
    double w, h;
public:
    Rectangle(double ww, double hh) : w(ww), h(hh) {}
    double area() const override { return w * h; }
};

int main() {
    // Shape s; // 错：抽象基类不能创建对象
    Shape* p = new Rectangle(3, 4);
    cout << p->area() << endl;
    delete p;
    return 0;
}
```

6. 虚拟继承

虚拟继承用于解决菱形继承中的“共同基类重复出现”问题。例如 B 和 C 都继承 A，D 又同时继承 B 和 C，如果不用 virtual，D 中可能有两份 A；使用 virtual 后，D 中只保留一份共享的 A。

```
#include <iostream>
using namespace std;

class A {
public:
    int x;
    A() : x(10) {}
};

class B : virtual public A {};
class C : virtual public A {};
class D : public B, public C {};

int main() {
    D d;
```

```
d.x = 20;    // 因为虚拟继承，D 中只有一份 A::x
cout << d.x << endl;
return 0;
}
□
```

五、模板、链表、栈、队列与 STL 容器

来源页码：第 16-25 页

1. 函数模板

函数模板是“函数的模具”。同一套代码可以用于 int、double、string 等不同类型。调用时，编译器根据实参类型生成具体函数。

- 定义格式：template <typename T> 或 template <class T>。
- T 是类型参数，也可以有普通参数。
- 函数模板调用时，先根据实参类型实例化。
- 模板函数调用时通常不进行实参到形参类型的自动转换。

```
#include <iostream>
using namespace std;

template <typename T>
T maxVal(T a, T b) {
    return a > b ? a : b;
}

int main() {
    cout << maxVal(3, 5) << endl;    // T 实例化为 int
    cout << maxVal(2.5, 1.8) << endl; // T 实例化为 double
    // cout << maxVal(3, 2.5); // 一般会出错：T 无法同时是 int 和 double
    cout << maxVal<double>(3, 2.5) << endl; // 显式指定 T 为 double
    return 0;
}
```

2. 类模板

类模板是“类的模具”。例如数组类 Array<T, N> 可以生成 int 数组、double 数组、char 数组等。类模板只有实例化后才是具体类。

- 实例化写法：类模板名<类型实参, 普通实参>。
- 普通实参通常是常量，如数组长度 N。
- 定义类模板的成员函数时，也要写 template 声明和 类名<T>::。

```
#include <iostream>
using namespace std;

template <typename T, int N>
class MyArray {
private:
    T data[N];
public:
    void set(int i, T value) { data[i] = value; }
    T get(int i) const { return data[i]; }
};

int main() {
    MyArray<int, 5> a;    // 类模板实例化为具体类
    a.set(0, 100);
    cout << a.get(0) << endl;

    MyArray<double, 3> b;
    b.set(1, 3.14);
    cout << b.get(1) << endl;
    return 0;
}
```

3. 链表类模板

链表是一种由节点组成的数据结构。每个节点保存数据和指向下一个节点的指针。和数组相比，链表插入删除方便，但按下标随机访问不方便。课件给出了 Node<T> 与 List<T> 的基本框架，并重点展示了有序插入。

- 节点模板 Node<T>：保存数据 num 和指针 next。
- 链表模板 List<T>：保存 head、tail、nodeCount，并提供 Insert、Remove、Find、Print。
- 插入节点时常见情况：空表、插到头部、插到尾部、插到中间。

```
#include <iostream>
using namespace std;

template <typename T>
class Node {
public:
    T num;
    Node<T>* next;
    Node(T n) : num(n), next(nullptr) {}
};

template <typename T>
class List {
private:
    Node<T>* head;
public:
    List() : head(nullptr) {}

    void InsertFront(T n) {
        Node<T>* tmp = new Node<T>(n);
        tmp->next = head;
        head = tmp;
    }

    void Print() const {
        Node<T>* p = head;
        while (p != nullptr) {
            cout << p->num << " ";
            p = p->next;
        }
        cout << endl;
    }
};

int main() {
    List<int> lst;
    lst.InsertFront(3);
    lst.InsertFront(2);
    lst.InsertFront(1);
    lst.Print(); // 1 2 3
    return 0;
}
```

4. 有序链表插入

课件第 21-23 页的链表 Insert 思路是：新建节点 tmp；如果链表为空，head 和 tail 都指向 tmp；否则判断是否插到头部、尾部或中间。

```
#include <iostream>
using namespace std;

template <typename T>
class Node {
public:
    T num;
    Node<T>* next;
    Node(T n) : num(n), next(nullptr) {}
};

template <typename T>
class SortedList {
private:
    Node<T>* head = nullptr;
    Node<T>* tail = nullptr;
public:
    void Insert(T n) {
```

```

Node<T>* tmp = new Node<T>(n);
if (head == nullptr) {
    head = tail = tmp;
    return;
}
if (n < head->num) {
    tmp->next = head;
    head = tmp;
    return;
}
if (n > tail->num) {
    tail->next = tmp;
    tail = tmp;
    return;
}
Node<T>* curr = head;
while (curr->next != nullptr) {
    if (curr->num <= n && curr->next->num > n) {
        tmp->next = curr->next;
        curr->next = tmp;
        return;
    }
    curr = curr->next;
}

void Print() const {
    for (Node<T>* p = head; p != nullptr; p = p->next) cout << p->num << " ";
    cout << endl;
}

};

int main() {
    SortedList<int> a;
    a.Insert(5); a.Insert(2); a.Insert(8); a.Insert(6);
    a.Print(); // 2 5 6 8
    return 0;
}

```

5. STL 容器：array、vector、list、forward_list、deque

容器就是“装数据的通用数据结构”。选择容器时，要看你更需要快速查找、快速插入删除，还是随机访问。

容器	通俗理解	适合场景	头文件
array	固定长度数组，长度编译期确定	元素个数固定，想要数组但更安全	<array>
vector	可自动扩容的动态数组	需要随机访问，不频繁在中间插入删除	<vector>
list	双向链表	频繁插入删除，不强调随机访问	<list>
forward_list	单向链表	更省空间，只需要单向遍历	<forward_list>
deque	双端队列，头尾插入删除较快	既要随机访问，又可能头尾操作	<deque>

```

#include <iostream>
#include <vector>
#include <list>
using namespace std;

int main() {
    vector<int> v = {1, 2, 3};
    v.push_back(4); // 尾部插入
    cout << v[2] << endl; // 随机访问快

    list<int> lst = {10, 20, 30};
    lst.push_front(5); // 头部插入方便
    lst.push_back(40);
    for (int x : lst) cout << x << " ";
    return 0;
}

```

```
}
```

6. 集合与映射：set、map、unordered_set、unordered_map

set 像“只存唯一元素的集合”；map 像“字典”，通过 key 找 value。unordered 版本底层通常是哈希表，平均查找很快，但元素没有排序。

- set/multiset：集合，set 不重复，multiset 允许重复，通常自动排序。
- map/multimap：键值对映射，map 的 key 不重复，multimap 允许重复 key。
- unordered_set/unordered_map：无序集合/映射，适合快速查找，不保证顺序。

```
#include <iostream>
#include <map>
#include <set>
using namespace std;

int main() {
    set<string> names;
    names.insert("China");
    names.insert("China"); // set 自动去重
    cout << names.size() << endl; // 1

    map<string, int> score;
    score["Tom"] = 90;
    score["Li"] = 85;
    cout << score["Tom"] << endl;
    return 0;
}
```

7. 栈与队列

栈和队列是常见数据结构，课件复习范围中提到。栈是“后进先出”，像叠盘子；队列是“先进先出”，像排队买票。C++ STL 中 stack 和 queue 是容器适配器。

- stack：push 入栈，pop 出栈，top 看栈顶。
- queue：push 入队，pop 出队，front 看队首，back 看队尾。

```
#include <iostream>
#include <stack>
#include <queue>
using namespace std;

int main() {
    stack<int> st;
    st.push(1); st.push(2); st.push(3);
    cout << st.top() << endl; // 3, 后进先出
    st.pop();

    queue<int> q;
    q.push(1); q.push(2); q.push(3);
    cout << q.front() << endl; // 1, 先进先出
    q.pop();
    return 0;
}
□
```

六、输入输出流与格式控制

来源页码：第 26-32 页

1. 流的概念与主要流类

流可以理解为“数据流动的管道”。输入流把数据从键盘/文件送入程序，输出流把数据从程序送到屏幕/文件。

- ios：输入输出流的基类。
- istream：输入流，如 cin。
- ostream：输出流，如 cout、cerr、clog。

- `iostream`: 输入输出流。
- `ifstream`: 文件输入流, 用于读文件。
- `ofstream`: 文件输出流, 用于写文件。
- `fstream`: 文件输入输出流, 可读可写。

2. 主流对象: `cin`、`cout`、`cerr`、`clog`

`cin` 用于标准输入, `cout` 用于标准输出, `cerr` 和 `clog` 用于错误/日志输出。`cerr` 通常不缓冲, 适合立即输出错误; `clog` 通常有缓冲。

```
#include <iostream>
using namespace std;

int main() {
    int age;
    cout << "请输入年龄: ";
    cin >> age;

    if (age < 0) {
        cerr << "错误: 年龄不能为负数" << endl;
    } else {
        clog << "日志: 输入成功" << endl;
        cout << "年龄是: " << age << endl;
    }
    return 0;
}
```

3. 提取运算符 `>>` 与插入运算符 `<<`

`>>` 是从输入流“提取”数据, `<<` 是向输出流“插入”数据。对于自定义类, 有时需要重载这两个运算符。

- `cin >> x`: 从键盘读入 `x`。
- `cout << x`: 把 `x` 输出到屏幕。
- 自定义类重载时常写成友元函数, 并返回 `istream&` 或 `ostream&`。

4. 格式控制函数与格式控制符

格式控制用于让输出更整齐。例如指定宽度、精度、填充字符。课件区分“格式控制函数”和“格式控制符”。

- 格式控制函数: 由流对象调用, 如 `cout.width(5)`、`cout.precision(3)`、`cout.fill('*')`。
- 格式控制符: 直接放入 `<<` 表达式, 如 `setw(5)`、`setprecision(3)`、`setfill('*')`, 需要包含 `<iomanip>`。
- `width/setw` 通常只对紧接着的下一次输出有效; `fill/setfill` 通常会持续影响后续输出, 直到重新设置。

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    int a = 42;
    double pi = 3.1415926;

    cout << setw(5) << a << endl;           // 输出宽度为 5
    cout << setfill('*') << setw(5) << a << endl; // 用 * 填充
    cout << setfill(" ") << fixed << setprecision(2) << pi << endl; // 3.14
    return 0;
}

#include <iostream>
using namespace std;

int main() {
    cout.width(6);           // 设置下一个输出宽度为 6
    cout.fill('#');          // 设置填充字符
    cout << "abc" << endl;   // ###abc

    cout.precision(4);       // 设置有效数字位数
    cout << 3.1415926 << endl;
    return 0;
}
```

```
}  
□
```

七、文件读写

来源页码：第 33-38 页，第 45-46 页例题

1. 文本文件与二进制文件

文本文件用字符形式保存，人可以用记事本直接看懂；二进制文件按内存字节保存，体积可能更小、读写更快，但人不能直接看懂。

- 文本文件：ASCII 字符方式，如保存“123”，文件中是字符 1、2、3。
- 二进制文件：二进制数方式，如保存整数 123，按 int 的字节表示保存。
- 文本文件常用 >>、<<、get、put、getline。
- 二进制文件常用 read、write。

2. 文件读写的一般过程

文件操作一般分三步：打开文件、读写文件、关闭文件。实际程序中还要检查文件是否成功打开。

```
#include <iostream>  
#include <fstream>  
using namespace std;  
  
int main() {  
    ofstream fout("data.txt"); // 打开文件用于写  
    if (!fout) {  
        cerr << "文件打开失败" << endl;  
        return 1;  
    }  
    fout << "hello file" << endl;  
    fout.close(); // 关闭文件  
  
    ifstream fin("data.txt"); // 打开文件用于读  
    string s;  
    getline(fin, s);  
    cout << s << endl;  
    fin.close();  
    return 0;  
}
```

3. 文本文件按字符读写：get 与 put

get 一次读一个字符，put 一次写一个字符。适合逐字符处理，比如复制文件、统计字符数。

```
#include <iostream>  
#include <fstream>  
using namespace std;  
  
int main() {  
    ifstream fin("input.txt");  
    ofstream fout("copy.txt");  
  
    char ch;  
    fin.get(ch);  
    while (!fin.eof()) {  
        fout.put(ch); // 把读到的字符写入新文件  
        fin.get(ch); // 再读下一个字符  
    }  
  
    fin.close();  
    fout.close();  
    return 0;  
}
```

易错提醒：更现代、更安全的写法是 while (fin.get(ch)) { ... }，可避免 eof 判断带来的边界问题。但考试若按课件写法，需看题目要求。

4. 文本文件按行读取：getline

getline 一次读取一整行，默认遇到

结束。适合读取比赛记录、学生信息、日志等一行一条记录的数据。

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main() {
    ifstream infile("MatchResult.txt");
    string line;

    while (getline(infile, line)) {
        cout << "读到一行: " << line << endl;
    }

    infile.close();
    return 0;
}
```

5. 二进制文件读写：read 与 write

read/write 按字节读写，参数通常需要转换成 char*。课件给出形式：fout.write((char*)(&ss), sizeof(ss));
fin.read((char*)(&ss), sizeof(ss));

```
#include <iostream>
#include <fstream>
using namespace std;

class Team {
public:
    char name[20];
    int points;
};

int main() {
    Team t = {"France", 6};

    ofstream fout("Standing.bin", ios::binary);
    fout.write((char*)&t, sizeof(t)); // 写入一个 Team 对象的字节
    fout.close();

    Team x;
    ifstream fin("Standing.bin", ios::binary);
    fin.read((char*)&x, sizeof(x)); // 读出一个 Team 对象的字节
    fin.close();

    cout << x.name << " " << x.points << endl;
    return 0;
}
```

6. 文件随机访问：seekg/seekp 与 tellg/tellp

随机访问就是不从头顺序读写，而是直接跳到文件某个位置。g 表示 get，即读指针；p 表示 put，即写指针。位置单位是字节。

- tellg：获取读指针位置。
- tellp：获取写指针位置。
- seekg：设置读指针位置。
- seekp：设置写指针位置。

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    fstream file("nums.bin", ios::in | ios::out | ios::binary | ios::trunc);
    int a[3] = {10, 20, 30};
    file.write((char*)a, sizeof(a));

    file.seekg(sizeof(int)); // 跳到第 2 个 int 的位置
    int x;
    file.read((char*)&x, sizeof(x));
    cout << x << endl; // 20
}
```

```

cout << "当前位置: " << file.tellg() << endl;
file.close();
return 0;
}

```

7. 综合例题思路：世界杯积分统计

课件第 45-46 页给出程序设计题：从 MatchResult.txt 读取比赛比分，计算球队积分，胜一场 3 分，平一场 1 分，负一场 0 分，并把 Team 数组以二进制写入 Standing.bin。

- 读取一行比赛结果，例如 France 0:0 Uruguay。
- 解析两个队名和比分。
- 按胜平负更新积分。
- 最后用 write 将 Team 数组写入二进制文件。

```

#include <iostream>
#include <fstream>
#include <sstream>
#include <map>
#include <vector>
#include <cstring>
using namespace std;

class Team {
public:
    char name[20];
    int points;
    Team() { name[0] = '\0'; points = 0; }
};

void OutPut_File(vector<Team>& teams) {
    ofstream fout("Standing.bin", ios::binary);
    for (auto& t : teams) {
        fout.write((char*)&t, sizeof(t));
    }
    fout.close();
}

int main() {
    ifstream fin("MatchResult.txt");
    map<string, int> points;
    string team1, score, team2;

    while (fin >> team1 >> score >> team2) {
        int pos = score.find(':');
        int s1 = stoi(score.substr(0, pos));
        int s2 = stoi(score.substr(pos + 1));

        if (s1 > s2) { points[team1] += 3; points[team2] += 0; }
        else if (s1 < s2) { points[team2] += 3; points[team1] += 0; }
        else { points[team1] += 1; points[team2] += 1; }
    }

    vector<Team> teams;
    for (auto& kv : points) {
        Team t;
        strcpy(t.name, kv.first.c_str(), 19);
        t.name[19] = '\0';
        t.points = kv.second;
        teams.push_back(t);
    }
    OutPut_File(teams);
    return 0;
}
□

```

八、课件例题答案与解析

来源页码：第 39-46 页

1.4 关于类和对象

正确答案：A。友元函数可以访问该类的私有数据成员。

- B 错：不同对象的普通数据成员通常各自占用内存；共享的是静态成员。
- C 错：类的公有成员函数可以访问本类 private 成员。
- D 在传统 C++ 课程语境中通常判错：一般要求通过构造函数或初始化列表初始化数据成员。现代 C++ 允许类内成员初始化，但考试应按课程体系判断。

1.5 构造函数初始化

可正确初始化的通常是 C 和 D。

- A 错：构造函数不能像普通成员函数那样由对象调用。
- B 错：对象数组不能这样统一带参数初始化。
- C 对：new TestClass(1,2) 调用有参构造函数。
- D 对：TestClass Obj4 = TestClass(1,2) 用临时对象初始化。

1.6 不能重载的函数

正确答案：D。析构函数不能重载，因为析构函数没有参数，一个类只能有一个析构函数。

1.7 继承与多态

更合理的答案是 D。函数重载要求函数名相同，但参数表必须不同，不能完全相同。

- A 不严谨：私有继承会把基类 public/protected 成员在派生类中变为 private，但基类 private 成员派生类不能直接访问。
- B 错：虚基类不是“只用来派生的类”，而是虚拟继承中的共享基类。
- C 错：友元关系不能继承。

1.9 函数模板和类模板

正确答案：D。函数模板可以重载，模板函数调用时通常不允许参数类型自动转换。

- A 错：类型参数可以是基本类型，也可以是类类型等。
- B 错：类模板的静态成员通常按具体实例化类型分别存在，不是在定义模板时直接创建所有成员。
- C 错：不能直接创建“类模板”的对象，必须先实例化成具体类，如 Box<int> b。

九、期末速记清单

1. 语法高频易错

- 类定义后必须加分号。
- 类外定义成员函数要写 类名::函数名。
- class 默认 private。
- 构造函数、析构函数没有返回值。析构函数不能重载。
- const 成员、引用成员、对象成员、基类构造常用初始化列表。
- 有指针成员时注意深拷贝：析构函数、拷贝构造、赋值运算符。
- 友元不是成员，但可以访问 private/protected。
- 成员 operator 左操作数是 this；友元 operator 参数包含全部操作数。

- 多态条件：基类指针/引用 + virtual 函数 + 派生类重写。
- 纯虚函数 =0；含纯虚函数的类是抽象基类，不能实例化。
- 类模板必须实例化后才能定义对象，如 List<int> list。
- setw 需要 <iomanip>，width/setw 通常只影响下一次输出。
- 二进制 read/write 参数常写 (char*)(&obj), sizeof(obj)。
- seekg/tellg 管读指针；seekp/tellp 管写指针。

2. 常见函数原型模板

```
// 友元方式重载比较运算符
friend bool operator>(const Time& a, const Time& b);

// 成员方式重载减法运算符
Time operator-(const Time& t) const;

// 赋值运算符重载
ClassName& operator=(const ClassName& other);

// 输入输出运算符重载
friend istream& operator>>(istream& in, ClassName& obj);
friend ostream& operator<<(ostream& out, const ClassName& obj);

// 函数模板
template <typename T>
T func(T a, T b);

// 类模板
template <typename T>
class List { /* ... */ };

// 二进制文件读写
fout.write((char*)(&obj), sizeof(obj));
fin.read((char*)(&obj), sizeof(obj));
```

C++ 期末复习知识点整理 | 根据用户上传课件整理